Normalizador estadístico en NASM: implementación escalar y vectorización manual con AVX2

Maria José Gamboa Quesada
Carnet: 2023071349

***Resumen*—El trabajo desarrolla y evalúa un normalizador estadístico de precisión simple para Linux x86-64. Se implementaron las funciones sum_array, compute_stats y normalize_array en NASM mediante dos alternativas: procesamiento escalar y SIMD de ocho floats con AVX2. Un driver C común proporciona lectura binaria, memoria alineada, validación, medición y exportación de resultados. La referencia independiente NumPy permite evaluar estadísticas y cada elemento normalizado. Los resultados muestran equivalencia dentro de la tolerancia relativa 1e-4 en los casos de prueba, manejo controlado de entradas inválidas y aceleración del núcleo vectorial en los cuatro tamaños medidos. En 50 millones de elementos se obtuvieron aproximadamente 158.21 ms para el escalar y 52.00 ms para AVX2, con un speedup de 3.04. Una corrección de acumulación por bloques permitió cumplir la tolerancia en los casos grandes manteniendo aritmética float32. La inspección GDB comprueba registros, alineación y salida; perf aporta evidencia complementaria sobre instrucciones, ciclos y comportamiento de caché.**

***Index Terms*—NASM, AVX2, SIMD, float32, normalización estadística, precisión, rendimiento.**

I. Introducción y objetivos

Se calculan suma, media, varianza poblacional, desviación estándar, mínimo y máximo de un arreglo float32, y se normaliza mediante y[i]=(x[i]-media)/sigma. Se compara un kernel NASM escalar, de un elemento por iteración, con otro NASM AVX2, de ocho elementos. Se estudian SIMD, alineación, reducciones, remanentes, ABI, precisión y rendimiento. El driver C gestiona archivos, memoria y tiempos.

I-A. Problema y motivación

La normalización estadística transforma una colección de muestras a una escala común y permite expresar cada observación respecto de la media de su conjunto. En señales o mediciones de sensores, las estadísticas descriptivas y el z-score constituyen operaciones frecuentes que recorren todos los datos. Para arreglos grandes, el coste de esos recorridos hace pertinente comparar un cálculo secuencial con uno que aproveche el paralelismo disponible dentro del procesador.

El problema no consiste únicamente en reemplazar una suma escalar por vaddps. La aplicación de una operación a varios datos se corresponde con el paralelismo de datos descrito por Patterson y Hennessy [1, cap. 3.7]. Culler, Singh y Gupta caracterizan este modelo como operaciones uniformes sobre arreglos bajo una secuencia de control común [2, sec. 1.3.5, pp. 56-57]. En este proyecto el paralelismo reside en los carriles de las instrucciones SIMD; el driver ejecuta una secuencia de llamadas y no distribuye tareas entre hilos. Hay que garantizar el contrato binario entre C y NASM, operar con memoria válida, reducir los resultados de distintos carriles, procesar tamaños no divisibles por ocho y controlar el efecto del redondeo. También se necesita una metodología que distinga una mejora real de una diferencia provocada por E/S o ruido temporal.

I-B. Objetivo general

Diseñar, implementar y evaluar dos núcleos NASM funcionalmente equivalentes para calcular estadísticas descriptivas y normalizar arreglos float32, comparando procesamiento escalar y vectorización manual AVX2 bajo un mismo contrato de entrada, salida, precisión y medición.

I-C. Objetivos específicos y criterios de evaluación

1. Implementar las tres interfaces del contrato de funciones respetando System V AMD64 ABI y separar el cálculo NASM de la gestión de recursos del driver C. La evidencia es la compilación de dos ejecutables con el mismo driver y firmas.
2. Calcular suma, media, varianza poblacional, desviación, mínimo, máximo y salida z-score en float32. La evidencia es la comparación automatizada con NumPy.
3. Desarrollar un camino AVX2 de ocho elementos, cargas alineadas, reducción horizontal y cola explícita. La evidencia se obtiene del código, las pruebas de remanentes, el diagrama y la inspección de registros y memoria con GDB.
4. Resolver N=0, N=1, sigma=0, negativos, extremos y archivos inválidos mediante políticas documentadas. La evidencia es el resultado de las suites de correctud.
5. Medir exclusivamente las llamadas a los kernels para los cuatro tamaños indicados, con al menos 30 repeticiones, media, desviación y gráfico de speedup.
6. Complementar la medida temporal con perf y explicar los límites del beneficio SIMD sin confundir IPC, ticks TSC, tiempo de kernel y tiempo del proceso completo.

I-D. Modelo matemático y alcance

Para N>0, las operaciones implementadas son:

$$S = \sum_{i=0}^{N-1} x_i, \qquad \mu = S/N, \tag{1}$$

$$v = \frac{1}{N} \sum_{i=0}^{N-1} (x_i - \mu)^2, \qquad \sigma = \sqrt{v}, \tag{2}$$

$$y_i = \begin{cases} (x_i - \mu)/\sigma, & \sigma > 0, \\ 0, & \sigma = 0. \end{cases} \tag{3}$$

La precisión simple IEEE 754 utiliza 32 bits: un bit de signo, ocho de exponente y 23 de fracción; los valores normalizados tienen un bit principal implícito [1, pp. 206-208]. Este formato justifica que la comparación tenga en cuenta el redondeo y no exija igualdad matemática exacta.

La varianza es poblacional: cada dato del arreglo pertenece al conjunto que se describe y se divide por N. Dividir por N-1 correspondería a un estimador muestral distinto del problema resuelto. Para sigma=0 se define una salida de ceros, una política operacional que evita una división matemáticamente indefinida. Para N=0 se produce error controlado en lugar de inventar estadísticas.

Como ejemplo de comprobación, x=[1,2,3,4] da S=10, mu=2.5, diferencias [-1.5,-0.5,0.5,1.5], cuadrados [2.25,0.25,0.25,2.25], v=1.25 y sigma aproximadamente 1.118034. La salida es aproximadamente [-1.3416408,-0.4472136,0.4472136,1.3416408]. Este caso permite relacionar las fórmulas con las instrucciones de resta, multiplicación, suma y división.

La implementación prioriza un algoritmo verificable, vectorización explícita y decisiones justificadas. El análisis se circunscribe a los datos y al equipo medidos; no presenta el speedup como propiedad universal ni demuestra exactitud para cualquier combinación posible de valores finitos.

II. Entorno y arquitectura

II-A. Plataforma, compilación y soporte SIMD

Las pruebas se realizaron en un procesador AMD Ryzen 5 4600G with Radeon Graphics, con soporte AVX2 confirmado en /proc/cpuinfo y lscpu. Se utilizaron NASM 3.01, GCC 15.2.0 y GDB 17.1.

Se usa Linux x86-64, precisión simple y System V AMD64. GCC compila el driver con -O2, -fno-tree-vectorize y -fno-tree-slp-vectorize; NASM ensambla los kernels con símbolos DWARF. No hay intrínsecos SIMD C ni vectorización automática en los kernels evaluados. El driver comprueba AVX2 antes de llamar a esa versión.

II-B. Responsabilidades de los módulos

El generador Python produce entradas deterministas. El driver C realiza la E/S, comprueba el formato, reserva arreglos alineados y entrega punteros a NASM. Cada ejecutable enlaza exactamente una biblioteca NASM: ambas exportan los mismos nombres, por lo que pueden reutilizar el contrato sin una selección dinámica de funciones. Enlazar las dos bibliotecas simultáneamente produciría símbolos duplicados y no corresponde al diseño de comparación adoptado.

La salida del driver se entrega al verificador, que calcula una referencia independiente. Los scripts de rendimiento reúnen las muestras y elaboran los resultados. Por tanto, Python no implementa el kernel evaluado ni participa en las llamadas temporizadas. Esta separación permite rastrear un fallo hasta entrada, interfaz, cálculo, salida o comprobación, en vez de mezclar etapas.

El archivo de arquitectura independiente es docs/arquitectura.pdf. Su fuente editable compatible con diagrams.net es docs/arquitectura.drawio. Describe módulos, comunicación ABI, flujo de control, registros y memoria. El contrato y su aplicación se detallan a continuación.

II-C. Formato binario, memoria y validaciones

La entrada es int32 little endian N seguido de N float32 little endian. El driver rechaza N<=0, tamaños inconsistentes y datos no finitos antes de calcular. posix_memalign reserva entrada y salida a 32 bytes. La salida contiene N floats normalizados sin encabezado. Se exportan estadísticas JSON para verificar.

Un archivo de entrada debe ocupar exactamente 4+4*N bytes; la salida ocupa 4*N bytes. El encabezado no se copia a la salida porque el consumidor conoce N desde la entrada. El driver detecta encabezados truncados, N no positivo, lecturas incompletas y bytes sobrantes. Las entradas NaN o infinitas se rechazan; si datos finitos generan estadísticas o salida no finitas, también se informa un error y se liberan los recursos.

La alineación expresa una propiedad de la dirección base: base modulo 32=0. No modifica el valor del float ni exige añadir relleno al archivo. El elemento i se direcciona como base+4*i. En el camino vectorial i avanza en ocho, de modo que cada dirección de grupo conserva la alineación: base+32*k. La cola usa accesos individuales y no lee más allá de los N elementos reservados.

El driver centraliza cierres y liberaciones en cleanup. Los punteros se inicializan a NULL y el código comprueba fallos antes de utilizar una reserva. Esta organización permite abandonar una operación incompleta sin duplicar liberaciones en cada condición. Las estadísticas se exportan adicionalmente en JSON, lo que facilita comparar valores completos y conservar las muestras.

II-D. Comunicación mediante System V AMD64 ABI

La convención utilizada corresponde a System V AMD64 ABI. sum_array recibe RDI=entrada y ESI=N y devuelve XMM0. compute_stats recibe RDI=entrada, ESI=N, RDX=media, RCX=varianza, R8=mínimo y R9=máximo, estos últimos como direcciones. normalize_array recibe RDI=entrada, RSI=salida, EDX=N, XMM0=media y XMM1=sigma. Los registros de argumentos enteros y floats se asignan independientemente. RBX, RBP y R12-R15 se preservan porque no se modifican; los kernels solo usan registros volátiles, sin llamadas ni pila.

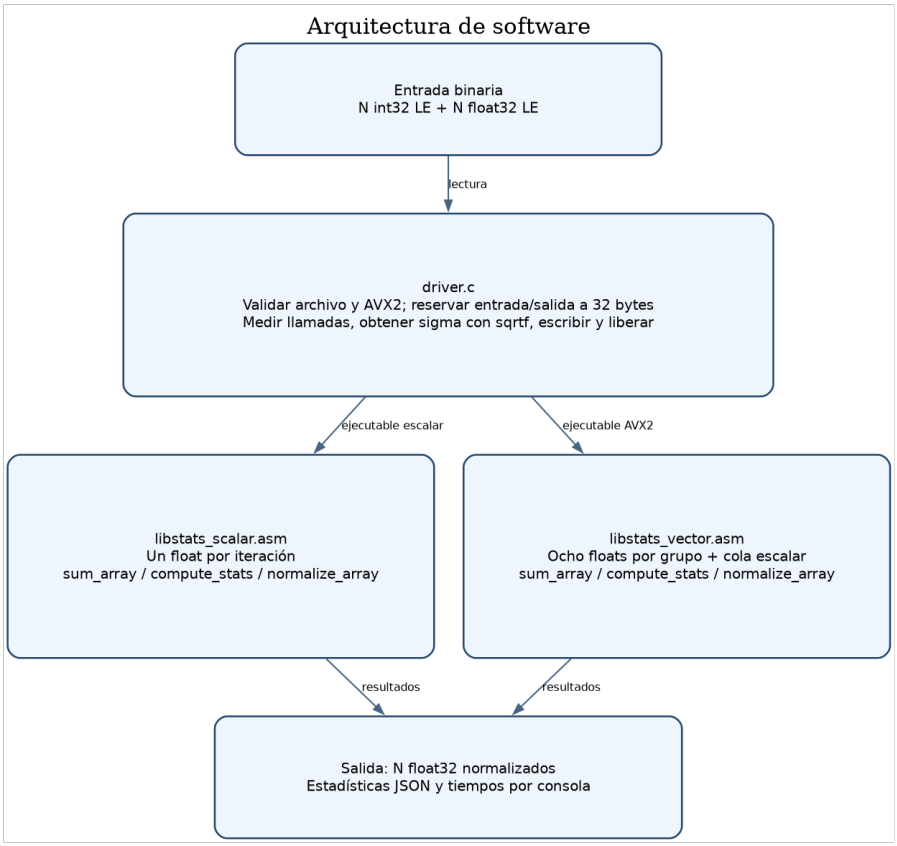


Figura 1. Arquitectura común y alternativas de cálculo. Fuente editable: arquitectura.drawio. El diagrama independiente amplía ABI, flujos, registros y memoria.

Las clases de argumentos se asignan por separado: en normalize_array los dos punteros y N usan registros enteros, mientras media y sigma ocupan XMM0/XMM1. No llegan en los registros correspondientes a la cuarta y quinta posición entera. compute_stats escribe sus cuatro resultados en las direcciones recibidas; no devuelve una estructura ni cuatro valores por el registro de retorno.

El código no modifica registros preservados por el callee ni realiza llamadas internas; por eso puede retornar sin un marco de pila. Los registros XMM/YMM usados son volátiles bajo esta ABI. Esta elección evita trabajo adicional de salvado y mantiene visible el estado de cada etapa en el depurador.

III. Implementación escalar

sum_array suma un float con addss por iteración. compute_stats realiza dos pasadas: suma/mínimo/máximo; después suma (x-media)^2 y divide por N. Se elige varianza poblacional, no muestral. normalize_array aplica subss/divss y escribe un float con movss. Para sigma=0 guarda ceros sin dividir. El driver obtiene sigma mediante sqrtf de la varianza devuelta: la interfaz de compute_stats entrega varianza y recibe sigma al normalizar.

III-A. Recorridos, estado e invariantes

sum_array inicializa el índice y el acumulador, incorpora input[i] y aumenta el índice hasta N. La dirección [rdi+rax*4] refleja que cada float ocupa cuatro bytes. La condición i<N es la que evita accesos fuera del arreglo. Las sumas parciales se incorporan al total cada 4096 elementos y el parcial final también se suma al salir; así se cubren bloques completos y el último bloque incompleto.

La primera pasada de compute_stats obtiene la suma y actualiza extremos. Mínimo y máximo comienzan con el primer dato válido. Inicializarlos a cero sería incorrecto para un arreglo completamente positivo o completamente negativo. El entero N se convierte a float antes de dividir y obtener la media.

La segunda pasada mantiene la media fija y acumula cuadrados de diferencias. No se usa la identidad E[x^2]-E[x]^2 porque puede restar cantidades grandes cercanas y sufrir cancelación. La opción de dos pasadas demanda otra lectura del arreglo, pero permite una interpretación directa de la fórmula y una vectorización simétrica. Su coste está incluido en ambas mediciones.

III-B. Fragmento de código y correspondencia matemática

Fragmento real de la segunda pasada escalar:

```
.second_pass:
    movss xmm5, [rdi + rax*4]  ; cargar x[i]
    subss xmm5, xmm0  ; restar la media
    mulss xmm5, xmm5  ; cuadrado de la diferencia
    addss xmm4, xmm5  ; acumular cuadrados
    inc eax
    test eax, 4095  ; detectar limite de 4096 elementos
    jnz .second_next
    addss xmm8, xmm4
    xorps xmm4, xmm4
.second_next:
    cmp eax, esi
    jl .second_pass
    addss xmm4, xmm8
    divss xmm4, xmm3  ; dividir por N: varianza poblacional
```

movss carga un valor en XMM5; subss resta la media XMM0; mulss eleva esa diferencia al cuadrado y addss la incorpora a XMM4. El índice aumenta uno, y cmp/jl repite mientras quedan elementos. test eax,4095 detecta un múltiplo de 4096: el parcial se añade a XMM8 y se reinicia. La división final por XMM3, que contiene N convertido a float, produce la varianza poblacional.

III-C. Normalización y contrato de precisión

La normalización conserva media y sigma y aplica una resta y una división por elemento, seguida de un almacenamiento. Primero distingue sigma nula para escribir ceros sin ejecutar divisiones. El driver comprueba que sigma y las estadísticas sean finitas antes de llamar a esta función. Por tanto, no se presenta el comportamiento de los kernels para NaN como parte de su contrato válido; los casos no finitos se gestionan en el límite de entrada y de cálculo.

Se usan solamente operaciones aritméticas float escalares. xorps inicializa registros a cero; no procesa varios elementos del arreglo en paralelo. Los tipos del driver son int, float, double para tiempos, enteros para TSC y punteros; FILE y timespec son interfaces de biblioteca para E/S y reloj.

IV. Implementación AVX2 y precisión

256 bits / 32 bits = ocho floats por YMM. La elección sigue el soporte AVX2 del equipo y el procesamiento de ocho float32 por grupo. El tratamiento de SSE y AVX en la bibliografía distingue registros de 128 y 256 bits, respectivamente, y sus capacidades de cuatro y ocho float32 [1, pp. 232-233]. La implementación concreta comprueba AVX2 antes de ejecutar sus instrucciones. SSE procesaría cuatro; AVX-512 requeriría otro soporte y otra implementación. La aritmética float empaquetada proviene de AVX; la difusión desde registro utilizada aquí requiere AVX2.

Cada iteración carga ocho valores con vmovaps. vaddps, vsubps, vmulps, vdivps, vminps y vmaxps operan por carriles. No se emplea FMA ni float64 en kernels. vbroadcastss repite media y sigma para normalizar todos los carriles. Al retornar se ejecuta vzeroupper para limpiar las mitades altas de YMM.

Patterson y Hennessy incluyen AVX2 entre las extensiones SIMD de x86 y describen su operación gather para cargas indexadas [1, sec. 6.3, p. 529]. Los arreglos de este proyecto se recorren de forma contigua, por lo que se emplean cargas ordinarias de 32 bytes, sin gather. Amadio también presenta AVX2 como una vía para vectorizar bucles [3, diap. 50]. Su ejemplo utiliza la opción de compilación -mavx2; aquí las instrucciones se escriben directamente en NASM. Estas referencias sustentan el contexto de vectorización, mientras que la elección de instrucciones y los valores de registros se verifican en el código y mediante la ejecución depurada.

IV-A. Organización de los carriles y control del bucle

El acumulador YMM contiene ocho sumas parciales, una por posición dentro de cada grupo. vaddps no entrega directamente la suma del arreglo: suma cada carril del grupo cargado con su carril correspondiente del acumulador. El índice y el límite siguen siendo enteros y los accesos a memoria se basan en el mismo arreglo usado por la versión escalar.

El límite N&-8 elimina los tres bits inferiores de N positivo, obteniendo 8*floor(N/8). De esta manera solo se ejecutan cargas YMM cuando existen ocho datos completos. El control de bloques de acumulación añade una reducción cada 4096 elementos, sin reiniciar los extremos ni omitir datos.

IV-B. Reducción horizontal de la suma

Fragmento real de la suma vectorial y su reducción:

```
.vector_loop:
    cmp eax, r10d
    jge .reduce
    vmovaps ymm1, [rdi + rax*4]  ; cargar ocho float32
     alineados
    vaddps ymm0, ymm0, ymm1  ; sumar por carril
    add eax, 8  ; avanzar ocho elementos
    test eax, 4095  ; detectar limite de 4096 elementos
    jz .reduce
    jmp .vector_loop
.reduce:
    ; Ocho carriles -> cuatro -> dos -> uno.
    vextractf128 xmm1, ymm0, 1  ; extraer cuatro carriles
     altos
    vaddps xmm0, xmm0, xmm1
    vhaddps xmm0, xmm0, xmm0
    vhaddps xmm0, xmm0, xmm0
    vaddss xmm8, xmm8, xmm0
    vxorps ymm0, ymm0, ymm0
    cmp eax, r10d
    jl .vector_loop
    vmovaps xmm0, xmm8
```

vextractf128 obtiene la mitad alta. Se suma a la baja y dos vhaddps reducen cuatro valores a uno. Se eligió esta secuencia por claridad y fácil inspección. Si los carriles contienen [a,b,c,d,e,f,g,h], la extracción y suma de mitades producen [a+e,b+f,c+g,d+h]. El primer vhaddps forma dos sumas de parejas; el segundo deja la suma de los ocho datos en el float bajo. Por ejemplo, [1,2,3,4,5,6,7,8] se reduce a [6,8,10,12], luego a [14,22,14,22] y finalmente a [36,36,36,36]. El retorno usa únicamente el valor bajo.

La extracción es necesaria porque una suma horizontal AVX en YMM actúa dentro de mitades de 128 bits; no combina automáticamente las ocho posiciones. Una alternativa sería una secuencia exclusiva de shuffles y sumas. Se eligió la combinación extracción/vhaddps por su trazabilidad y correspondencia visible con el árbol de reducción, sin introducir optimizaciones opcionales.

Una reducción transforma un arreglo en un único resultado. Kirk y Hwu describen la suma y los extremos mediante combinaciones por etapas [4, sec. 6.1, pp. 99-100]. Ese patrón

explica el árbol horizontal utilizado aquí: ocho parciales se combinan hasta obtener uno. Se toma el principio algorítmico de reducción; la ejecución concreta utiliza registros SIMD de CPU y no requiere kernels CUDA.

La separación entre cálculo parcial y combinación final también aparece en el producto interno de Bisseling [5, sec. 1.3, pp. 9-12]. La suma puede interpretarse como el producto interno del arreglo con un vector de unos; la suma de desviaciones cuadradas, como el producto interno de las desviaciones consigo mismas. En esta implementación los parciales se mantienen en carriles y bloques, sin comunicación entre procesadores ni sincronizaciones BSP. La analogía permite justificar la descomposición del cálculo sin aplicar al programa un modelo de costos de una arquitectura distinta.

IV-C. Extremos y normalización empaquetada

Para mínimo y máximo se combinan mitades, luego vpermilps intercambia parejas con 0x4e y adyacentes con 0xb1; vminps/vmaxps incorporan cada intercambio. Se inicializan los extremos con el primer valor, evitando sesgos hacia cero.

Las máscaras 0x4e y 0xb1 seleccionan respectivamente las posiciones [2,3,0,1] y [1,0,3,2]. Al comparar con esas permutaciones, el extremo se propaga entre parejas separadas y luego entre vecinos. A diferencia de una suma, esta reducción utiliza min/max y no agrega magnitudes. La semilla del primer dato es válida también cuando no existe ningún grupo completo.

En normalize_array, vbroadcastss replica media y sigma en los ocho carriles. Una carga, resta, división y almacenamiento procesan un grupo completo. La rama sigma=0 almacena el registro de ceros en grupos de ocho y termina con almacenamientos escalares para los elementos sobrantes.

IV-D. Remanente y seguridad de los accesos

R10D=N&-8 es el límite de bloques completos. N=15 procesa 0-7 con YMM y 8-14 con instrucciones escalares. N=7 utiliza solo la cola. Las bases están alineadas y cada avance vectorial es 32 bytes; el escalar avanza 4 bytes.

Para N=8 se procesa exactamente un grupo; para N=16, dos grupos; para N=1 se ejecuta solamente la cola y luego la política de varianza nula. Estas transiciones son parte del algoritmo, no excepciones improvisadas por el driver. La cola mantiene el acumulador reducido y agrega únicamente los elementos que quedan, por lo que no introduce padding ni valores artificiales.

IV-E. Desarrollo y corrección de la acumulación float32

La primera versión perdió precisión: N=100000 dio varianza 1.55632997, frente a 1.55859375 de NumPy, fuera de tolerancia. Para corregirlo ambas versiones suman bloques de 4096 y acumulan sus parciales en XMM8, siempre float32. test eax,4095 detecta el cierre de bloque; el bloque incompleto se incorpora al final. Suma y cuadrados usan esta estrategia. Los mínimos y máximos se conservan entre bloques. Esta corrección no busca optimizar velocidad ni introduce compensación compleja. No garantiza exactitud para todos los datos.

El redondeo se vuelve relevante cuando el acumulador crece y una contribución pequeña queda por debajo de su resolución. La suma flotante no es asociativa: cambiar la agrupación cambia los redondeos, tal como se discute en la comparación de sumas de precisión simple [1, p. 243]. Acortar los parciales a 4096 datos reduce la longitud de la cadena; en AVX2 cada carril incorpora hasta 512 datos antes de la reducción del bloque. El total entre bloques sigue siendo float32.

El valor 4096 es una potencia de dos y múltiplo de ocho, por lo que permite un control simple con máscara y no introduce una cola adicional en los límites. La elección se validó contra el fallo observado; no constituye un valor óptimo universal. Usar float64 habría alterado el requisito de precisión de cálculo; una acumulación compensada aumentaría complejidad. La solución mantiene el algoritmo explícito y documenta que aún puede fallar ante otras distribuciones con cancelación o magnitudes extremas.

V. Pruebas de correctud

La referencia usa NumPy float32, varianza poblacional y salida cero si sigma=0. Se comparan todas las estadísticas y todos los elementos normalizados contra la referencia y entre versiones. Error relativo <=1e-4; para referencia exactamente cero se usa error absoluto <=1e-6, porque el relativo no se define. Las mediciones del error usan float64 en Python, sin cambiar la referencia.

V-A. Diseño de datos y criterio de aceptación

La generación usa semilla 2026 para los tamaños pequeños y valores positivos múltiplos de 1/8, exactamente representables en binario. Se añaden arreglos negativos, constantes y extremos finitos para activar comportamientos que una entrada positiva aleatoria no ejercita. N=0 comprueba rechazo, N=1 la varianza nula, y N=7/8/15/16 las transiciones entre cola y grupos completos.

Los tamaños grandes adicionales emplean grupos de 16 valores [-3,4,0,...,0]. Cada grupo suma 1; la media es 1/16 y la varianza poblacional 1.55859375. Este patrón limita el crecimiento de la suma y permite estudiar el procesamiento en float32 en 50 millones de elementos. No representa una muestra aleatoria ni permite generalizar el rendimiento a todas las señales posibles.

La tolerancia relativa se evalúa como |obtenido-referencial/|referencial|. Cuando la referencia es exactamente cero se usa el límite absoluto declarado. Las estadísticas, la longitud de salida y todos los elementos se verifican; la comprobación no se limita a que el programa termine o a revisar unos valores. Una diferencia fuera de tolerancia genera un estado de fallo del verificador.

V-B. Casos obligatorios: referencia y salidas obtenidas

Los siguientes resultados provienen de results/correctness_vector.json. Las tablas muestran la referencia y las dos salidas reales, con redondeo solo para presentación; las comparaciones usan los valores completos. Los archivos de entrada originales y la semilla 2026 permiten reproducirlos.

Tabla I
RESULTADOS: INPUT_1.DAT

Estadístico	NumPy	Escalar	AVX2
Suma	1	1	1
Media	1	1	1
Varianza	0	0	0
Sigma	0	0	0
Mínimo	1	1	1
Máximo	1	1	1

Tabla II
RESULTADOS: INPUT_7.DAT

Estadístico	NumPy	Escalar	AVX2
Suma	18.375	18.375	18.375
Media	2.625	2.625	2.625
Varianza	1.066964	1.066964	1.066964
Sigma	1.03294	1.03294	1.03294
Mínimo	0.875	0.875	0.875
Máximo	4	4	4

Tabla III
RESULTADOS: INPUT_8.DAT

Estadístico	NumPy	Escalar	AVX2
Suma	12.875	12.875	12.875
Media	1.609375	1.609375	1.609375
Varianza	2.05249	2.05249	2.05249
Sigma	1.432652	1.432652	1.432652
Mínimo	0.125	0.125	0.125
Máximo	4	4	4

Tabla IV
RESULTADOS: INPUT_15.DAT

Estadístico	NumPy	Escalar	AVX2
Suma	35	35	35
Media	2.333333	2.333333	2.333333
Varianza	0.7555554	0.7555555	0.7555555
Sigma	0.8692269	0.8692269	0.8692269
Mínimo	0.625	0.625	0.625
Máximo	3.75	3.75	3.75

Tabla V
RESULTADOS: INPUT_16.DAT

Estadístico	NumPy	Escalar	AVX2
Suma	42.875	42.875	42.875
Media	2.679688	2.679688	2.679688
Varianza	1.71283	1.71283	1.71283
Sigma	1.308751	1.308751	1.308751
Mínimo	0.25	0.25	0.25
Máximo	4	4	4

Tabla VI
RESULTADOS: INPUT_1000.DAT

Estadístico	NumPy	Escalar	AVX2
Suma	2099.125	2099.125	2099.125
Media	2.099125	2.099125	2.099125
Varianza	1.32244	1.322438	1.32244
Sigma	1.149974	1.149973	1.149974
Mínimo	0.125	0.125	0.125
Máximo	4	4	4

V-C. Caso input_0.dat: PASS

Entrada: N=0, sin floats. Esperado: error controlado, sin salida. scalar: código 1, ERROR: encabezado incompleto o N <= 0. vector: código 1, ERROR: encabezado incompleto o N <= 0.

V-D. Caso input_1.dat: PASS

Entrada: N=1. Datos positivos pseudoaleatorios, múltiplos de 1/8, semilla 2026.

Normalización completa: PASS contra NumPy y entre versiones.

V-E. Caso input_7.dat: PASS

Entrada: N=7. Datos positivos pseudoaleatorios, múltiplos de 1/8, semilla 2026.

Normalización completa: PASS contra NumPy y entre versiones.

V-F. Caso input_8.dat: PASS

Entrada: N=8. Datos positivos pseudoaleatorios, múltiplos de 1/8, semilla 2026.

Normalización completa: PASS contra NumPy y entre versiones.

V-G. Caso input_15.dat: PASS

Entrada: N=15. Datos positivos pseudoaleatorios, múltiplos de 1/8, semilla 2026.

Normalización completa: PASS contra NumPy y entre versiones.

V-H. Caso input_16.dat: PASS

Entrada: N=16. Datos positivos pseudoaleatorios, múltiplos de 1/8, semilla 2026.

Normalización completa: PASS contra NumPy y entre versiones.

V-I. Caso input_1000.dat: PASS

Entrada: N=1000. Datos positivos pseudoaleatorios, múltiplos de 1/8, semilla 2026.

Normalización completa: PASS contra NumPy y entre versiones.

V-J. Caso input_negative.dat: PASS

Entrada: N=16. Valores -1,-2,...,-16.

Normalización completa: PASS contra NumPy y entre versiones.

Tabla VII
RESULTADOS: INPUT_NEGATIVE.DAT

Estadístico	NumPy	Escalar	AVX2
Suma	-136	-136	-136
Media	-8.5	-8.5	-8.5
Varianza	21.25	21.25	21.25
Sigma	4.609772	4.609772	4.609772
Mínimo	-16	-16	-16
Máximo	-1	-1	-1

Tabla VIII
RESULTADOS: INPUT_CONSTANT.DAT

Estadístico	NumPy	Escalar	AVX2
Suma	40	40	40
Media	2.5	2.5	2.5
Varianza	0	0	0
Sigma	0	0	0
Mínimo	2.5	2.5	2.5
Máximo	2.5	2.5	2.5

Tabla IX
RESULTADOS: INPUT_EXTREMES.DAT

Estadístico	NumPy	Escalar	AVX2
Suma	8e+18	7.999999e+18	8e+18
Media	5e+17	5e+17	5e+17
Varianza	7.499999e+35	7.499999e+35	7.499999e+35
Sigma	8.660254e+17	8.660253e+17	8.660254e+17
Mínimo	-1e+18	-1e+18	-1e+18
Máximo	1e+18	1e+18	1e+18

V-K. Caso input_constant.dat: PASS

Entrada: N=16. Dieciséis valores iguales a 2,5; sigma=0 y salida de ceros.

Normalización completa: PASS contra NumPy y entre versiones.

V-L. Caso input_extremes.dat: PASS

Entrada: N=16. [-1e18,1e18,1e18,1e18] repetido cuatro veces, redondeado a float32.

Normalización completa: PASS contra NumPy y entre versiones.

V-M. Cobertura adicional y pruebas grandes

La suite pequeña tiene 62 casos, de los cuales 62 pasaron. Además de los anteriores, incluye N=1..33 para todas las colas, constantes de N=7/15, extremos en el último elemento de N=15, límites 4095/4096/4097/8191/8192/8193 y entradas malformadas, NaN, infinito, desbordamiento y archivo inexistente.

Verificación grande, incluida la salida normalizada elemento por elemento:

Las pruebas de límites de bloque comprueban tanto el cierre exacto como un elemento antes y después. Las pruebas de mínimo/máximo en el último elemento verifican que la cola participa en las estadísticas, y no solamente en la suma. Los archivos inválidos permiten confirmar que no se normalizan datos truncados, NaN o infinitos y que el mensaje de error llega acompañado de un código no cero.

Tabla X
VERIFICACIÓN COMPLETA DE LOS CUATRO TAMAÑOS DE RENDIMIENTO

N	Resultado	Máximo error relativo estadístico por versión
1000	PASS	scalar: 1.17e-06, vector: 2.72e-07
100000	PASS	scalar: 1.34e-10, vector: 1.34e-10
1000000	PASS	scalar: 1.34e-10, vector: 1.34e-10
50000000	PASS	scalar: 1.34e-10, vector: 1.34e-10

La corrección se verificó con los mismos tamaños y el mismo límite de error; no se relajó la tolerancia para aceptar los resultados.

VI. RENDIMIENTO

VI-A. Protocolo experimental y variables medidas

Se midieron tres intervalos CLOCK_MONOTONIC exclusivos de las llamadas NASM, sumados por repetición; sqrtf, E/S y validación quedan fuera. Se ejecutó un calentamiento descartado y 30 repeticiones por versión y tamaño, sin descartar atípicos. La comparación usa el mismo trabajo y entradas para ambas versiones, consistente con la función de un benchmark como marco de comparación [6, sec. 11.7.1]. Los arreglos se reutilizan. El proceso se fijó a la CPU lógica indicada en los metadatos. La desviación es muestral (Nrepeticiones-1). sum_array calcula su suma y compute_stats la repite para ser independiente; ambas versiones hacen ese mismo trabajo. Los ticks TSC no son cycles de perf.

La suma temporal corresponde al trabajo total de las tres funciones por repetición, no a una sola pasada. Cada intervalo también incluye el pequeño coste de instrumentación que rodea la llamada; en N pequeño su proporción es más importante. CLOCK_MONOTONIC evita cambios de hora civil y las diferencias se convierten de segundos/nanosegundos a milisegundos.

Los ticks se capturan mediante lfence/rdtsc y se acumulan por diferencias. El TSC avanza a frecuencia de referencia, mientras el contador cycles de perf observa un evento hardware distinto. Esta separación evita identificar ticks con ciclos efectivos del núcleo cuando cambia la frecuencia de ejecución.

Para r repeticiones se calcula media=sum(t)/r y desviación muestral sqrt(sum((t-media)^2)/(r-1)). No debe confundirse ese divisor con el N de la varianza poblacional de los datos. Se reporta speedup=media_escalar/media_AVX2; una razón mayor que uno indica reducción del tiempo en la versión vectorial.

VI-B. Resultados temporales

CPU lógica utilizada: 0. Archivos: benchmark.csv, benchmark_samples.csv y benchmark_metadata.json.

Gráfico: results/speedup.png y speedup.pdf disponibles.

VI-C. Interpretación del speedup y variabilidad

N=100000 presenta una dispersión escalar del 27.2 % del promedio; su speedup no demuestra una ventaja estable de

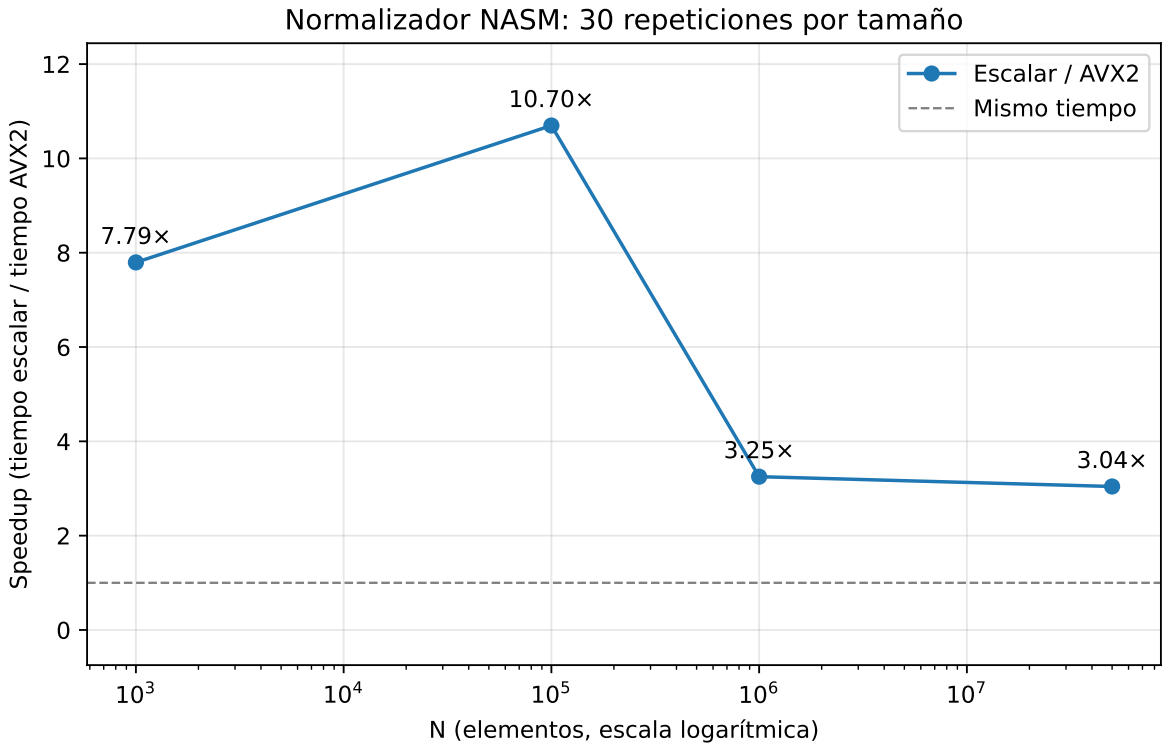


Figura 2. Speedup calculado a partir de los promedios, con N en escala logarítmica. La dispersión escalar en N=100000 requiere interpretar ese punto junto con la tabla temporal.

Tabla XI
TIEMPO DEL NÚCLEO: MEDIA Y DESVIACIÓN MUESTRAL DE 30 REPETICIONES

N	Escalar ms +/- desv.	AVX2 ms +/- desv.	Speedup
1000	0.00516453 +/- 4.48e-05	0.0006626 +/- 2.29e-05	7.794x
100000	0.392705 +/- 0.107	0.0367117 +/- 0.00113	10.697x
1000000	3.0684 +/- 0.0208	0.943914 +/- 0.0202	3.251x
50000000	158.211 +/- 0.589	51.9997 +/- 0.439	3.043x

esa magnitud. Ocho carriles no son un techo universal de 8x: cuentan también dependencias e instrucciones. Para N=50 millones el speedup observado ronda 3x y los arreglos ocupan 400 MB. Es compatible con límites de memoria, sin demostrar que sean el único factor. Frecuencia, tareas del sistema y otros hilos no se controlaron. N=1000 usa datos pseudoaleatorios y los mayores un patrón; eso limita la comparación entre tamaños. La mejora medida solo cubre kernels, no el programa con E/S. La Ley de Amdahl explica cómo los costes no acelerados limitan la mejora total.

El máximo observado no se interpreta como prueba de que ocho carriles produzcan siempre ocho veces más rendimiento. La dependencia del acumulador es distinta, cambia el número de instrucciones por dato y las condiciones de caché no son iguales entre tamaños. La localidad espacial de arreglos y la diferencia entre niveles de la jerarquía proporcionan el marco conceptual para examinar esa dependencia del tamaño [1, pp. 388-390]. Magoulès, Roux y Houzeaux muestran que aumentar las unidades de cálculo no basta si la memoria no puede suministrar los operandos, y explican la transferencia por líneas de caché [7, pp. 6, 8-10]. Este principio justifica analizar conjuntamente el trabajo aritmético y los accesos contiguos; la identificación del cuello de botella concreto requiere mediciones adicionales, no se deduce solo del speedup. En el caso de alta dispersión, el promedio escalar puede estar afectado por actividad externa. No se repitió el experimento hasta conseguir un resultado deseado ni se descartaron muestras para elevar la aceleración; se comunica la variabilidad que efectivamente se midió.

En los tamaños mayores hay una mejora más próxima a tres veces. El trabajo incluye varios recorridos de entrada, escritura de salida, divisiones y reducciones. Cada repetición realiza nominalmente cinco transferencias de arreglo: lectura en sum_array, dos lecturas en compute_stats y lectura/escritura en normalize_array. Para N=50 millones eso representa unos 1000 MB de accesos lógicos. Es una descripción del algoritmo, no una medición de tráfico físico a DRAM, porque caché y política de escritura pueden modificar ese tráfico.

El tamaño conjunto de entrada/salida es 400 MB en el caso mayor. La reducción de instrucciones no reduce en igual pro-

Tabla XII
CONTADORES PERF DEL PROCESO COMPLETO: N=1000000

Evento o indicador	Escalar	AVX2
cycles	431859146	147613656
instructions	989500505	167848544
cache-misses	389026	564126
cache-references	20932635	20328108
IPC	2.291	1.137
Fallos de caché (%)	1.858	2.775

porción los bytes que se deben visitar. Esto es coherente con una contribución importante de memoria, pero no identifica por sí solo la caché o el recurso limitante. Para demostrar saturación harían falta contadores de tráfico o un experimento de ancho de banda específico.

La Ley de Amdahl relaciona el beneficio total con la fracción que se mejora [1, pp. 49-50]. Por esa ley, si una fracción p del tiempo total se acelera por un factor s, el speedup global sería 1/((1-p)+p/s). La lectura/escritura de archivos queda fuera del tiempo reportado; no se dispone de una medida de p del programa completo y por ello no se calcula una aceleración global ficticia.

VI-D. Contadores hardware y comparación de IPC

perf incluye todo el proceso, no solo los intervalos del CSV. Se intentó cycles,instructions,cache-misses,cache-references. IPC=instructions/cycles; tasa de fallos=cache-misses/cache-references, con eventos dependientes de CPU.

Contadores reales: N=1000000, 30 repeticiones y un calentamiento.

La captura externa obtuvo contadores después de habilitar temporalmente perf; se restauró después la restricción. Los archivos perf_scalar.txt y perf_vector.txt contienen las salidas reales. AVX2 ejecutó 5.90 veces menos instrucciones y consumió 2.93 veces menos ciclos de proceso. Su IPC menor no implica menor rendimiento: una instrucción SIMD procesa varios elementos. Las referencias de caché son similares y la tasa de fallos vectorial es mayor. Esto muestra que reducir instrucciones no elimina el tráfico de memoria; no permite identificar por sí solo una caché concreta ni demostrar saturación de ancho de banda. Los contadores son del tamaño 1 millón y no se extrapolan como contadores medidos para 50 millones. Los tiempos del CSV y los ciclos de proceso tienen distinto alcance y no se igualan directamente.

Amadio distingue la recolección de contadores con perf stat y define IPC como instrucciones por ciclo [3, diaps. 22, 26-27]. Por ello se presentan juntos el número de instrucciones, los ciclos y el tiempo: comparar solo IPC omitiría cuántos elementos procesa cada instrucción de las dos implementaciones.

VI-E. Validez y límites del experimento

La bibliografía distingue la validación funcional de la evaluación del rendimiento y advierte sobre interferencia y aislamiento de experimentos [6, secs. 11.7.4-11.7.5]. La afinidad fija la CPU lógica, pero no controla completamente frecuencia, temperatura, interrupciones o actividad de otros hilos. El calentamiento reduce costes de primera ejecución y describe una condición de reutilización que debe considerarse al interpretar resultados. Las muestras de N=1000 tienen tiempos muy cortos y mayor sensibilidad relativa al coste de medición.

La diferencia de distribución entre entradas pequeñas y grandes también impide atribuir cada cambio del gráfico únicamente a la jerarquía de caché. El análisis conserva esa limitación en lugar de presentar una causalidad que el diseño experimental no permite demostrar. Los hashes del driver y kernels asocian las mediciones a las fuentes entregadas: cambiar los kernels exigiría volver a medir, no reutilizar automáticamente esta tabla.

VII. EVIDENCIA GDB

Se preparó una sesión para N=16 con breakpoint antes de vaddps, stepi, print $ymm0.v8_float, comprobación de alineación y x/16fw tras normalizar. make evidence-gdb prepara y ejecuta las instrucciones de inspección. El objetivo de la sesión es relacionar una operación SIMD concreta con el contenido real de los carriles, verificar las bases alineadas y observar el arreglo normalizado después de completar la función. La transcripción real se conserva en docs/evidencia_gdb/vector_16.txt.

Fragmento de la evidencia real:

```
YMM0 inmediatamente despues de vaddps:
$1 = {0.25, 3, 3, 3.75, 3.5, 0.75, 3.25, 4}
Breakpoint 2 at 0x5555555574be: file src/libstats_vector.asm
    , line 169.

Breakpoint 2, normalize_array () at src/libstats_vector.asm
    :169
169         test edx, edx

Alineacion de entrada y salida (restos esperados: 0):
$2 = 0
$3 = 0
main (argc=<optimized out>, argv=<optimized out>) at src/
    driver.c:146
146             ticks_after = read_ticks();

Salida despues de retornar de normalize_array:
0x55555555c2c0: -1.85649323     0.244746685     0.244746685
          0.817812085
0x55555555c2d0: 0.626790285     -1.47444952     0.435768485
          1.00883389
0x55555555c2e0: -1.28342772     0.626790285     1.00883389
           0.435768485
0x55555555c2f0: -0.423829645    0.531279385     0.913322985
          -1.85649323
```

La evidencia procede de una sesión externa de GDB. Se distingue la captura funcional de las mediciones de rendimiento.

Después del primer vaddps se espera que los ocho carriles coincidan con los datos de entrada de la semilla 2026, como confirma la transcripción. Para inspeccionar la salida se guarda RSI al entrar a normalize_array porque es volátil, y finish vuelve al driver después de normalizar. GDB no se usa para medir tiempos; el estudiante debe practicar stepi/nexti y explicar cada paso.

El acumulador se inicializó en cero, por lo que después de la primera suma cada carril contiene el dato de entrada correspondiente. El registro mostrado no debe confundirse con la suma total 42.875, que resulta de incorporar el segundo

grupo y realizar la reducción. Los restos de alineación observados, ambos cero, concuerdan con la condición exigida por vmovaps.

La inspección de memoria muestra 16 valores, por lo que incluye ambos grupos normalizados. En particular, el primer dato 0.25, con media 2.6796875 y sigma aproximadamente 1.30875111, produce -1.85649323, como aparece en la transcripción. El archivo de salida de esta sesión se comparó adicionalmente contra NumPy. Esta concordancia enlaza observación de bajo nivel y verificación numérica. Las direcciones impresas pertenecen a esa ejecución y no deben reutilizarse como constantes al repetir una sesión con otras reservas o disposición ASLR.

VIII. Conclusiones, limitaciones y trabajo futuro

Las dos versiones calculan las estadísticas y normalización y pasan los casos pequeños y los cuatro tamaños grandes con la tolerancia pedida. AVX2 conserva la alineación y maneja todas las colas. La acumulación por bloques corrigió un fallo real de precisión sin abandonar float32. Las medidas muestran una mejora vectorial, con variación importante en un tamaño y límites de alcance.

VIII-A. Respuesta a los objetivos del proyecto

El objetivo de equivalencia funcional se cumple en los casos evaluados: las tres interfaces NASM producen estadísticas y normalización aceptadas por la referencia independiente. El objetivo de vectorización manual se concreta en cargas alineadas YMM, operaciones de ocho carriles, reducciones explícitas y remanentes escalares; no se delegó el cálculo a intrínsecos ni al compilador.

Los casos borde se resolvieron mediante un contrato visible: N=0 y archivos inválidos se rechazan, sigma=0 produce ceros, y negativos, extremos finitos y tamaños no divisibles por ocho pasan las comprobaciones. Los límites de bloque y todas las colas recibieron pruebas adicionales, lo que respalda las transiciones de control y no solamente el resultado de un tamaño cómodo.

El objetivo experimental se cumple con cuatro tamaños, 30 muestras por alternativa, resumen estadístico y gráfico. La mejora observada confirma la utilidad de SIMD en este núcleo; la reducción del beneficio para datos grandes y la variabilidad de un tamaño impiden sustituir el análisis por un supuesto 8x teórico. perf y GDB aportan evidencias de naturaleza distinta y complementaria: reducción de instrucciones y ciclos de proceso, y correspondencia entre instrucciones, registros y memoria.

La corrección de precisión muestra que vectorizar correctamente requiere atender tanto al flujo como al redondeo. Los bloques hicieron pasar los casos grandes sin relajar la tolerancia ni abandonar float32, manteniendo las mismas operaciones estadísticas y un diseño fácil de inspeccionar.

VIII-B. Limitaciones y líneas de continuación

El diseño no promete precisión para todos los datos finitos: puede desbordar o perder precisión ante magnitudes extremas o cancelación. El patrón grande reduce diversidad estadística y las capturas perf cubren únicamente un tamaño. No se midió ancho de banda físico ni se aislaron todas las fuentes de ruido. La comparación temporal corresponde al núcleo; no representa el tiempo total incluyendo almacenamiento y lectura desde disco.

Como continuación se propone evaluar distribuciones con signos mixtos y medias cercanas a cero, comparar estrategias de acumulación bajo el mismo contrato, repetir experimentos con control de frecuencia y carga, y usar contadores específicos de memoria. Estas propuestas responden a límites observados; no son funcionalidades que se atribuyan a la implementación entregada.

Atribución del uso de herramientas

Se utilizó Codex y agentes de código como asistencia para implementar módulos, revisar contratos, automatizar pruebas y preparar documentación. La integración se verificó mediante compilación, comparación numérica y revisión de evidencias. Las decisiones técnicas y sus alternativas se explican en este informe; la asistencia de IA se declara conforme al requisito de integridad académica.

Referencias

[1] D. A. Patterson y J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, ARM Edition. Morgan Kaufmann, 2017. Pasajes citados: cap. 1, pp. 49–50 (ley de Amdahl); sec. 3.5, pp. 206–208 (precisión simple IEEE 754); sec. 3.7, pp. 232–233 (SIMD, SSE y AVX); sec. 6.3, pp. 524–529 (paralelismo de datos y AVX2); sec. 3.10, p. 243 (no asociatividad de la suma flotante); sec. 5.1, pp. 388–390 (localidad y jerarquía de memoria).

[2] D. Culler, J. P. Singh y A. Gupta, *Parallel Computer Architecture: A Hardware/Software Approach*, borrador preliminar. Morgan Kaufmann, 1997. Pasaje citado: sec. 1.3.5, pp. 56–57 (paralelismo de datos y organización SIMD).

[3] G. Amadio, "Performance Analysis and Optimization on Linux," EP-SFT, material de presentación, 17 ago. 2022. Pasajes citados: diap. 22 (IPC); diap. 26–27 (contadores con perf stat); diap. 50 (vectorización SIMD mediante AVX2).

[4] D. B. Kirk y W.-m. W. Hwu, *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2010. Pasaje citado: sec. 6.1, pp. 99–100 (reducciones de suma, mínimo y máximo mediante combinación por etapas).

[5] R. H. Bisseling, *Parallel Scientific Computation: A Structured Approach Using BSP*, 2.ª ed. Oxford University Press, 2020. Pasaje citado: sec. 1.3, pp. 9–12, algoritmo 1.1 (producto interno mediante resultados parciales y combinación final).

[6] P. E. McKenney, ed., *Is Parallel Programming Hard, And, If So, What Can You Do About It?*, versión v2023.06.11a, jun. 2023. Pasajes citados: sec. 11.7.1, pp. 224–225 (comparación mediante benchmarks); secs. 11.7.4–11.7.5, pp. 225–226 (errores de medición, interferencia y aislamiento).

[7] F. Magoulès, F.-X. Roux y G. Houzeaux, *Parallel Scientific Computing*. ISTE y John Wiley & Sons, 2016. Pasajes citados: sec. 1.1, p. 6 (balance entre operaciones y accesos a memoria); sec. 1.2.2, pp. 8–10 (jerarquía, líneas de caché y patrones de acceso). La paginación corresponde a los números impresos de cada obra; en la presentación se indican las diapositivas.